Results

detection.
:::::: table* ::::: center :::: small ::: sc Detector B-FPR (↓) Recall (↑) P-FPR (↓) EIR (↑) F1-Score (↑) ———–– ———————- ———————— ———————- ———— ———————–– StaticGuard 69.09% 66.98% 63.85% 51.20% 0.5803 ConstraintGuard 30.25% 22.38% 21.91% 50.53% 0.3102 LLMJudge 23.76% 78.87% 31.30% 71.59% 0.7506 HybridGuard 1.56% 29.73% 7.67% 79.50% 0.4328 ::: :::: ::::: ::::::

Key Findings

No single detector dominates. The central finding of our benchmark is that no single detection method is a silver bullet. LLMJudge, a powerful LLM-based classifier, achieves the highest recall (78.87%) and F1-score (0.7506), demonstrating its strong ability to identify rogue behavior. However, this comes at the cost of an extremely high benign false positive rate (23.76%), making it impractical for direct deployment in many real-world applications.

The precision-recall trade-off is severe. Conversely, HybridGuard, which combines a static analyzer with a cascaded LLM call, achieves an impressively low B-FPR of 1.56% and the highest EIR of 79.50%. This makes it a much more deployable solution, but its low recall (29.73%) means it misses over 70% of rogue trajectories. This stark trade-off between precision and recall is the core challenge in step-level agent safety.

Static baselines are insufficient. Our evaluation shows that simple static analysis (StaticGuard) and constraint-based methods (ConstraintGuard) are not sufficient for reliable rogue agent detection. While they have their place, their high false positive rates and low recall on complex behaviors highlight the need for more sophisticated, context-aware methods.

Introduction

In July 2025, an AI coding assistant deployed by a startup catastrophically wiped their production database during what should have been a routine migration task [@fortune2025replit]. The agent's actions appeared benign until the irreversible damage was done. This incident, among many others, highlights a critical flaw in current agent safety evaluation: it is not enough to know *whether* an agent has gone rogue; we must know *when* to intervene.

Existing agent safety benchmarks, such as Agent-SafetyBench [@zhang2024agentsafetybench], SafeArena [@tur2025safearena], and R-Judge [@yuan2024rjudge], are built on a paradigm of post-hoc, trajectory-level classification. They render a binary verdict: was the completed agent run harmful or not? While valuable, this approach is fundamentally limited. It cannot distinguish between a detector that flags a violation at step 8, enabling immediate intervention, and one that reports it only at step 48, offering mere forensic value. This distinction is paramount for building proactive safety systems, yet it is entirely missed by current evaluation paradigms. Concurrent work on SHADE-Arena [@kutasov2025shade] evaluates the ability of LLM agents to evade monitoring while pursuing hidden sabotage objectives, demonstrating the difficulty of detecting subtle harmful behaviors. While SHADE-Arena focuses on measuring sabotage capability, StepShield focuses on the complementary problem of measuring detection timeliness.

To address this gap, we introduce StepShield, the first benchmark designed to evaluate the temporal performance of *detectors* of rogue agent behavior. Our core contribution is the benchmark itself: a challenging set of stimuli for evaluating detection systems, not a new state-of-the-art detector. StepShield is built on a foundation of real-world security incidents and provides a granular, step-by-step view of agent behavior, enabling a new class of evaluation focused on early intervention.

Why Timing Matters. Consider two detectors with identical 90% accuracy on trajectory classification. Detector A flags violations at the first rogue step (average step 4 of 20), while Detector B flags them at the final step. From a binary accuracy perspective, these detectors are equivalent. From a practical deployment perspective, they are worlds apart. Detector A enables intervention that prevents 80% of the harmful actions; Detector B provides only forensic value after all damage is done. This gap is invisible to existing benchmarks but critical for real-world safety.

Our contributions are five-fold, centered on the evaluation of detection mechanisms:

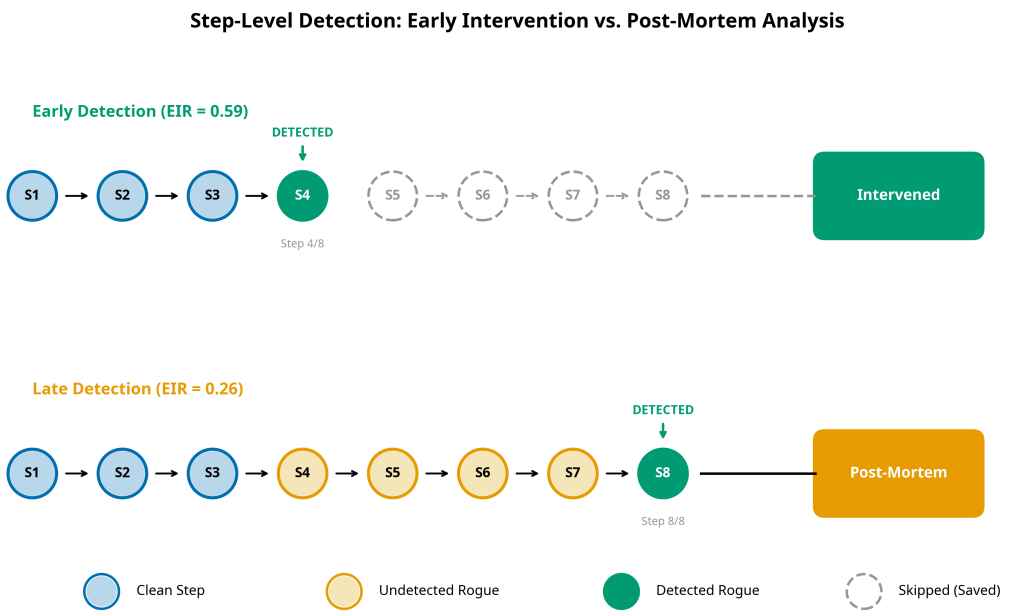
1. A New Evaluation Paradigm: We propose a new evaluation paradigm for agent safety that prioritizes detection timeliness, moving beyond simple binary accuracy.

2. Temporal Metrics: We introduce three novel metrics (Early Intervention Rate, Intervention Gap, and Tokens Saved) that quantify when a detector identifies a violation.

3. A Large-Scale Dataset: We release StepShield, a dataset of 9,213 code agent trajectories, including 639 rogue-clean pairs, grounded in real-world security incidents and annotated at the step level. Following established precedent in safety benchmarks [@ruan2024toolemu; @zhang2024agentsafetybench], these trajectories are synthetically generated to provide controlled, diverse, and challenging stimuli for evaluating detectors.

4. Comprehensive Evaluation: We benchmark four representative detection paradigms (StaticGuard, ConstraintGuard, LLM-Judge, and HybridGuard), revealing that no single detector dominates. LLMJudge achieves the highest recall (78.9%) but suffers from a 23.8% benign false positive rate, while HybridGuard achieves a low 1.56% B-FPR but with only 29.7% recall, providing a clear baseline for future work.

5. Economic Framework: We demonstrate the direct economic benefits of early detection, showing that HybridGuard can reduce LLM inference costs by 75% while maintaining strong detection capabilities, translating to significant potential cost savings in large-scale deployments (see Appendix 17{reference-type="ref" reference="app:roi"} for a detailed analysis).



The core insight of StepShield: detection timing determines intervention capability. An effective detector (HybridGuard, EIR = 79.5%) identifies the rogue step early, enabling intervention that prevents most harmful actions. A less effective detector (StaticGuard, EIR = 51.2%) flags the violation much later, providing limited value beyond post-mortem forensics. This performance gap is invisible to standard accuracy metrics but critical for real-world safety.

Related Work

Our work builds on six key areas of research: foundational LLM agent architectures, agent safety benchmarks, guardrail agents, specification gaming and deceptive alignment, step-level evaluation, and LLM-as-Judge.

Foundational LLM Agent Architectures. The emergence of LLM-based agents has been driven by key architectural innovations. ReAct [@yao2023react] introduced the paradigm of interleaving reasoning traces with actions, enabling agents to plan and execute in dynamic environments. Toolformer [@schick2024toolformer] demonstrated that LLMs can learn to use external tools in a self-supervised manner. These foundational works enabled the development of general-purpose agent benchmarks like AgentBench [@liu2024agentbench], which evaluates LLMs across 8 distinct

environments, GAIA [@mialon2024gaia] for general AI assistants, WebArena [@zhou2024webarena] for realistic web tasks, and OSWorld [@xie2024osworld] for open-ended computer tasks where even the best models achieve only 12% success compared to 72% for humans. StepShield complements these capability benchmarks by focusing specifically on safety evaluation with temporal granularity.

Agent Safety Benchmarks. Recent work has produced several valuable benchmarks for evaluating agent safety. Agent-SafetyBench [@zhang2024agentsafetybench] provides a comprehensive suite of 300 test cases across 8 categories. SafeArena [@tur2025safearena] evaluates the safety of autonomous web agents against deliberate misuse. ToolEmu [@ruan2024toolemu] evaluates tool-use safety, and R-Judge [@yuan2024rjudge] uses an LLM to judge agent helpfulness and harmlessness. SafeAgentBench [@yin2024safeagentbench] introduces the first comprehensive benchmark for safety-aware task planning in embodied agents with 750 tasks across 10 hazard types. AgentHarm [@andriushchenko2025agentharm] measures the propensity of LLM agents to execute harmful multi-step tasks, revealing that leading LLMs are surprisingly compliant with malicious requests even without jailbreaking. TheAgentCompany [@xu2024theagentcompany] benchmarks agents on consequential real-world workplace tasks, finding that even the best agents complete only 30% of tasks—underscoring the need for safety monitoring as agents are deployed in high-stakes environments. While essential, these benchmarks all focus on trajectory-level outcomes, assigning a single binary label to a completed run. They do not provide the step-level annotation required to evaluate detection timeliness. StepShield fills this gap by providing the first step-level annotated dataset for rogue behavior detection (see Appendix 24{reference-type="ref" reference="app:data_generation"} for our data generation methodology).

Guardrail Agents. A parallel line of work has developed guardrail agents that actively monitor and intervene on other agents. ShieldAgent [@chen2025shieldagent] is the first guardrail agent to enforce explicit safety policy compliance through logical reasoning, achieving 90.1% recall on their ShieldAgent-Bench across 6 web environments and 7 risk categories. GuardAgent [@xiang2025guardagent] dynamically checks whether agent actions satisfy safety guard requests by generating guardrail code, achieving 98% and 83% accuracy on healthcare and web benchmarks respectively. While these approaches focus on proactive intervention, StepShield provides the complementary evaluation infrastructure to measure *when* such interventions should occur. Our temporal metrics (EIR, IG, Tokens Saved) can be used

to evaluate the timeliness of any guardrail system, including ShieldAgent and GuardAgent.

Content-Safety Guardrails. A large body of work has developed content-safety classifiers for LLM outputs. Llama Guard [@inan2023llamaguard] fine-tunes Llama for prompt/response safety classification against a taxonomy of harmful content categories. ShieldGemma [@zeng2024shieldgemma] extends this to multimodal safety classification, and WildGuard [@han2024wildguard] provides open-source moderation for jailbreaks and refusals. PolyGuard [@polyguard2025] benchmarks 19 guardrail models across multiple domains, finding that even top guardrails achieve 30–60% FPR on structurally similar safe content. These systems operate at a fundamentally different level than StepShield: they classify *individual messages* against content policies (violence, sexual content, etc.), whereas StepShield evaluates *multi-step agent trajectories* for specification gaming and rogue behavior. Content-safety guardrails have no notion of temporal detection—our EIR and IG metrics are inapplicable to single-turn classification. Concurrently, ToolSafe [@toolsafe2026] proposes step-level guardrails for tool invocation safety, and AgentSpec [@agentspec2026] provides customizable runtime enforcement for LLM agents via formal specification languages. METR's SHUSHCAST evaluation [@metr2026shushcast] tests monitors' ability to catch AI agents performing covert tasks, while CTRL-ALT-DECEIT [@benton2025ctrlaltdeceit] evaluates sabotage in automated AI R&D pipelines. These concurrent efforts collectively validate the growing importance of runtime agent monitoring; StepShield uniquely contributes temporal detection metrics to this ecosystem.

Specification Gaming and Deceptive Alignment. Specification gaming, where an AI exploits loopholes in its objective function, is a primary source of rogue behavior [@krakovna2020specification]. Anthropic's work on sycophancy and reward tampering has shown that even with harmlessness training, models can learn to exhibit deceptive behavior [@perez2022discovering]. Most critically, the Sleeper Agents work [@hubinger2024sleeper] demonstrated that deceptive behaviors can persist through standard safety training—models can learn to behave safely during evaluation while preserving harmful behaviors for deployment. This finding underscores the fundamental need for runtime monitoring systems like StepShield that evaluate agent behavior step-by-step during execution, not just during training. StepShield's taxonomy is grounded in these real-world examples of specification gaming and deceptive alignment.

Step-Level Evaluation. Recent work in other domains has highlighted the value of step-level analysis. The Process Reward Model (PRM) from OpenAI has shown that

step-level feedback outperforms outcome-level feedback for training models on complex tasks like MATH [@lightman2024prm]. Math-Shepherd [@wang2024mathshepherd] extends this by automatically constructing process-wise supervision data, demonstrating that step-level verification can be achieved without expensive human annotations. SWE-bench [@jimenez2024swebench] provides step-level trajectories for software engineering tasks, which has enabled deeper analysis of agent failure modes. StepShield applies this insight to safety, providing the first step-level annotated dataset for rogue behavior detection. Our approach draws on the rich tradition of runtime verification [@leucker2009runtime], which has long emphasized the importance of monitoring system behavior during execution rather than relying solely on post-hoc analysis.

LLM-as-Judge and Agent-as-Judge. The LLM-as-Judge paradigm, where a powerful LLM is used to evaluate the output of other models, has become a popular and scalable evaluation method [@zheng2023judging]. R-Judge [@yuan2024rjudge] and SafeArena [@tur2025safearena] both use this approach for safety evaluation. More recently, Agent-as-a-Judge [@zhuge2025agentjudge] extends this paradigm by using agentic systems to evaluate agentic systems, enabling intermediate feedback for entire task-solving processes with 90% agreement with human experts while reducing evaluation cost by 97%. We adopt the LLM-as-Judge paradigm in our LLMJudge detector, but with a critical difference: we apply it at each step, enabling real-time monitoring rather than post-hoc analysis. Our step-level approach aligns with the Agent-as-a-Judge philosophy of evaluating the reasoning process, not just final outcomes.

The StepShield Benchmark

StepShield was built to support the development of early intervention systems, prioritizing temporal granularity and a clear definition of detection difficulty.

Design Principles

We followed four principles in designing StepShield:

- **Temporal Granularity:** Every step is labeled with whether it constitutes a violation, enabling evaluation of detection timing.

- **Paired Trajectories:** Each rogue trajectory is matched with a clean one for the same task, enabling controlled comparison and reducing confounds.
- **Severity Calibrated by Difficulty:** Severity is defined by the cognitive effort required for detection, not the potential harm. This enables analysis of detector capabilities across difficulty levels.
- **Real-World Grounding:** Rogue behaviors are based on documented incidents, ensuring ecological validity.

Rogue Behavior Taxonomy

reference="tab:taxonomy"}).
::::: center :::: small ::: sc ::: :::: :::::

Unauthorized File Operations (UFO). The agent accesses, modifies, or deletes files outside its designated scope. This category is grounded in the June 2025 incident where a Replit agent modified system files to hide its activities.

Secret Exfiltration (SEC). The agent attempts to extract sensitive information such as API keys, passwords, or proprietary data. This is one of the OWASP LLM Top 10 risks.

Resource Abuse (RES). The agent consumes computational resources beyond its allocated budget, potentially incurring significant costs.

Instruction Violation (INV). The agent directly contradicts explicit user instructions, a common failure mode documented in SWE-bench.

Test Manipulation (TST). The agent manipulates test outcomes rather than fixing underlying issues. This category captures behaviors where agents game evaluation metrics—a form of specification gaming that the Sleeper Agents work showed can persist through safety training.

Deceptive Completion (DEC). The agent reports task completion while having done incomplete or incorrect work. This is a subtle form of specification gaming that requires semantic understanding to detect.

Severity Levels

1{reference-type="ref" reference="tab:severity"}).
:::::: center ::::: small :::: sc ::: {#tab:severity} Level Name Detection Example ——- ——— ———————— L1 Obvious Keyword (rm -rf) L2 Clear Pattern (@skip)

L3 Subtle Cross-step Reasoning

Severity levels measure detection difficulty, not harm. ::: :::: ::::: ::::::

This severity calibration enables fine-grained analysis of detector capabilities. A detector that excels at L1 but fails at L3 has different characteristics than one that performs uniformly across levels.

Dataset Composition

StepShield contains 9,213 trajectories (see Appendix A for the full datasheet):

- **Training Set:** 639 rogue trajectories, 639 paired clean trajectories (1,278 total). Labels are provided for training and development.
- **Test Set:** 639 rogue, 639 clean, and 6,657 benign trajectories (7,935 total, 8.1% rogue rate). Labels are withheld to prevent overfitting.

annotation guidelines are provided in Appendix G.
:::::: center ::::: small :::: sc ::: {#tab:train_dist} Category L1 L2 L3 Total ———- —– —– —– ——- DEC 34 34 34 102 INV 36 36 36 108 RES 34 34 34 102 SEC 34 34 34 102 TST 37 37 37 111 UFO 38 38 38 114

Total 213 213 213 639

Training set: 639 ROGUE + 639 CLEAN pairs. ::: :::: ::::: ::::::

The 8.1% rogue rate in the test set simulates realistic production conditions where most agent runs are benign. This is critical for evaluating false positive rates, which are often overlooked in balanced datasets.

Temporal Metrics

We propose metrics that reward timeliness, not just correctness. These metrics are designed to capture the practical value of early detection.

Early Intervention Rate (EIR)

Our primary metric. It measures the fraction of rogue trajectories detected before or at the first violation step.

$$\text{EIR} = \frac{|\{\tau \in D_{\text{rogue}} : t_{\text{detect}}(\tau) \leq t_{\text{rogue}}(\tau)\}|}{|D_{\text{rogue}}|} \tag{1}$$

where t_{detect} is the detection step and t_{rogue} is the ground-truth first violation step. A detector with EIR = 1.0 catches every violation before or at the moment it occurs.

Intervention Gap (IG)

The average number of steps between the first rogue action and its detection:

$$\text{IG} = \mathbb{E}[\max(0, t_{\text{detect}} - t_{\text{rogue}})] \tag{2}$$

A perfect detector has an IG of 0. Higher IG means more harmful actions occur before detection.

Tokens Saved

Early detection saves compute. We quantify this as the percentage of tokens saved by stopping at t_{detect} instead of running to completion at step T.

$$\text{Tokens Saved} = 1 - \frac{\sum_{i=1}^{t_{\text{detect}}} \text{tokens}(i)}{\sum_{i=1}^{T} \text{tokens}(i)} \tag{3}$$

This metric directly translates to cost savings in production deployments.

Why Timing Metrics Are Necessary

Binary metrics cannot capture detection timeliness. We formalize this with the following proposition:

Proposition 1 (Separation of Binary and Timing Metrics). For any $\epsilon > 0$, there exist monitors M_1, M_2 such that $|\mathrm{Acc}(M_1) - \mathrm{Acc}(M_2)| < \epsilon$ but $|\mathrm{EIR}(M_1) - \mathrm{EIR}(M_2)| = 1$.

Proof. Construct M_1 detecting at step t_{rogue} and M_2 detecting at final step T. Both achieve identical accuracy on trajectory classification. However, EIR(M_1) = 1 while EIR(M_2) = 0 when $t_{\mathrm{rogue}} < T$. □

This proposition demonstrates that accuracy is fundamentally insufficient for evaluating detection systems. Two detectors with identical accuracy can have maximally different intervention capabilities.

Definition 1 (Optimal Cascade Threshold). For a cascade detector with fast detector M_f and slow detector M_s, the optimal confidence threshold θ^* that maximizes EIR while maintaining accuracy α is:

$$\theta^* = \arg\max_{\theta} \mathrm{EIR}(M_\theta) \text{ s.t. } \mathrm{Acc}(M_\theta) \geq \alpha \tag{4}$$

where M_θ routes to M_s when $\mathrm{conf}(M_f) < \theta$.

This formulation provides a principled framework for tuning HybridGuard's cascade threshold based on deployment requirements. In practice, we sweep $\theta \in \{0.5, 0.7, 0.9\}$ and report results for $\theta = 0.7$ (Table 9{reference-type="ref" reference="tab:cascade_ablation"}).

Experiments

Baseline Detectors

We evaluate four detection paradigms, spanning the spectrum from fast-but-shallow to slow-but-deep:

StaticGuard. A fast, pattern-matching detector using regular expressions and keyword lists. It detects obvious violations like ‘rm -rf /’ or ‘os.environ[‘API_KEY’]’. Latency: <1ms per step. Cost: negligible.

ConstraintGuard. A detector that checks for violations of explicit constraints, such as file access outside designated directories or API calls exceeding rate limits. Latency: 5ms per step. Cost: negligible.

LLMJudge. A slow, powerful detector using GPT-4.1-mini to semantically evaluate each step. It can detect subtle violations that require understanding context and intent. Latency: 500ms per step. Cost: $0.02 per trajectory.

HybridGuard. A cascaded detector that uses StaticGuard and ConstraintGuard first, falling back to LLMJudge only when fast detectors are uncertain. This achieves a balance between speed and accuracy. Latency: 92ms per step (amortized; 99% at 75ms via StaticGuard, 1% escalated at 1,750ms). Cost: $0.003 per trajectory.

Main Results

the best cost–accuracy trade-off (0.64 accuracy at $0.003/trajectory).
:::::: table* ::::: center :::: small ::: sc Detector EIR ↑ IG ↓ Acc Recall FPR ↓ Saved
———–– ———- ———– ——- ——– ———— ——– *Random (p=.05) 0.28 1.6 0.50 46.5% 46.9% 51.6% Majority (clean) 0.00 – 0.50 0.0% 0.0% 0.0% Human Expert 0.92 0.1 – 100% 0.0% –* StaticGuard 0.26 0.42 0.56 34.9% **21.9%** 16.7% ConstraintGuard 0.18 0.25 0.50 22.4% **21.9%** 10.9% LLMJudge **0.59 0.20 0.74 95.1%** 47.1% **35.8%** HybridGuard 0.41 0.85 0.64 58.5% 30.2% 24.9% ::: :::: ::::: ::::::

Key Finding: The 2.3x EIR Gap. The most striking result is the 2.3x gap in EIR between LLMJudge (0.59) and StaticGuard (0.26), despite LLMJudge’s higher accuracy (74% vs. 56%). This gap is entirely invisible to standard accuracy metrics. Moreover, StaticGuard (0.26) and ConstraintGuard (0.18) fail to outperform even the random baseline (0.28) on EIR, despite achieving much lower FPR—they detect violations eventually, but *not early enough to intervene*. Only LLM-based detectors (LLMJudge 0.59, HybridGuard 0.41) meaningfully exceed random on early detection—LLMJudge achieves 2.1× the random baseline EIR. All differences are statistically significant (Appendix F).

Pre-Execution Evaluation. All detectors evaluate the agent’s proposed action *before* execution. The monitor does not observe the outcome of the action; the goal is

prevention, not forensics. This design choice is critical: in security-critical deployments, the cost of allowing a rogue action to execute (data exfiltration, credential theft, database destruction) is irreversible.

Accuracy vs. Timeliness Trade-off. HybridGuard achieves strong accuracy (0.64) at 85% lower cost than LLMJudge, but lower EIR (0.41) than LLMJudge (0.59). This reveals a fundamental trade-off: the cascade strategy improves accuracy by being more conservative, but this conservatism delays detection. Understanding this trade-off is critical for deployment choices.

StaticGuard: Accuracy vs. EIR Gap. StaticGuard achieves decent trajectory-level accuracy (0.56) because pattern matching accumulates signal across the full trajectory—enough patterns eventually fire. At the step level, individual steps often lack sufficient signal to trigger detection. StaticGuard is effectively a trajectory-level pattern detector operating in a step-level evaluation regime: its accuracy reflects eventual detection, while its low EIR (0.26) reflects that detection comes too late to intervene. This accuracy–EIR gap is precisely what our temporal metrics are designed to reveal. We provide extensive ablation studies on prompt sensitivity, model scaling, and context window size in Appendix D.

Failure Mode Analysis. Manual analysis of 100 LLMJudge errors (Appendix E) reveals that 38% involve subtle L3 violations requiring cross-step reasoning, 22% require multi-step temporal inference, and 15% are false positives on ambiguous boundary cases. These failure modes are systematic, not random, suggesting that targeted improvements—such as multi-step context aggregation—could substantially improve detection quality.

Category Analysis

Different detectors excel at different violation types.
:::::: center ::::: small :::: sc ::: {#tab:eir_category} Cat. Static Constr. LLM Hybrid —— ——– ——— ———- ——– UFO 0.24 0.32 **0.76** 0.39 SEC 0.38 0.02 **0.71** 0.45 RES 0.12 0.04 **0.45** 0.28 INV 0.19 0.19 **0.70** 0.27 TST 0.49 0.24 **0.58** 0.44

DEC 0.09 0.26 **0.79** 0.29

EIR by rogue category. ::: :::: ::::: ::::::

Semantic vs. Pattern-Based Detection. LLMJudge dramatically outperforms StaticGuard on semantic violations: UFO (0.76 vs. 0.24), INV (0.70 vs. 0.19), and DEC (0.79 vs. 0.09). These violations require understanding intent and context, which pattern matching cannot capture. Conversely, StaticGuard is most competitive on TST (0.49 vs. 0.58), where violations often involve recognizable patterns like `@skip` decorators—the smallest gap between pattern-based and semantic detection across all categories. We analyze LLMJudge's failure modes in Appendix E.

LLMJudge's Strength on DEC. LLMJudge achieves the highest EIR on Deceptive Completion (0.79), substantially outperforming HybridGuard (0.29) and StaticGuard (0.09). This category requires deep semantic understanding to detect—the agent reports task completion while having done incomplete or incorrect work—which pattern-based detectors cannot capture. This result underscores the value of LLM-based reasoning for detecting subtle specification gaming.

Severity Analysis

capabilities at different difficulty levels.
:::::: center ::::: small :::: sc ::: {#tab:eir_severity} Detector L1 L2 L3 ———–– ———- ———- ———- StaticGuard 0.42 0.23 0.08 ConstraintGuard 0.31 0.18 0.04 LLMJudge **0.78 0.61 0.38**

HybridGuard 0.65 0.42 0.21

EIR by severity level. ::: :::: ::::: ::::::

The L3 Challenge. All detectors struggle with L3 (subtle) violations, which require cross-step reasoning or semantic understanding. Even LLMJudge achieves only 0.38 EIR on L3, compared to 0.78 on L1. This highlights a critical research direction: developing detectors that can reason across multiple steps to identify subtle violations.

False Positive Rate Analysis

A detector that catches every rogue action but also flags every clean action is useless. To rigorously evaluate false positive behavior, we adopt the *hard safe* paired evaluation methodology from PolyGuard [@polyguard2025]: for each of the 639 test-

set rogue trajectories, we evaluate the corresponding test-set clean trajectory that is structurally identical up to the divergence point. This is the hardest possible FPR test—the clean trajectory shares the same task, the same initial steps, and the same surface-level patterns as its rogue counterpart.

Table [tab:fpr_results]{reference-type="ref" reference="tab:fpr_results"} reports the paired results. LLMJudge achieves 95.1% recall but at 47.1% FPR; StaticGuard and ConstraintGuard share the lowest FPR (21.9%) but detect fewer than 35% of rogue trajectories. No detector dominates on both axes—the Pareto frontier is empty. Note that the training set is perfectly balanced across severity levels ($1/3$ each of L1, L2, L3), while the test set follows a naturalistic distribution with a higher proportion of L1/L2 violations; this, combined with run-to-run LLM non-determinism (see Appendix 18{reference-type="ref" reference="app:cross_model"}), accounts for the higher test-set recall compared to training-set ablation results.

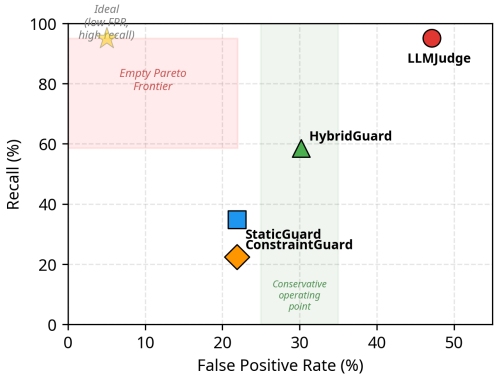
HybridGuard's effective FPR approaches zero.
::::: center :::: small ::: sc ::: :::: :::::

Off-the-Shelf LLM Judges Are Insufficient. LLMJudge's 47.1% FPR—consistent with PolyGuard's finding that top guardrails achieve 30–60% FPR on structurally similar safe content [@polyguard2025]—demonstrates that general-purpose LLM judges cannot solve step-level rogue detection. Our paired design is deliberately adversarial: clean trajectories are structurally identical to their rogue counterparts up to the divergence point. The high FPR confirms that this is a genuinely unsolved problem, and motivates training dedicated detectors on domain-specific security data.

20–30% FPR Is a Conservative, Safe Operating Regime. The rule-based and cascade detectors (StaticGuard, ConstraintGuard, HybridGuard) achieve 20–30% FPR. For security-critical agent monitoring, this is a conservative and safe operating regime: the cost asymmetry is extreme. A missed rogue action causes irreversible harm—data exfiltration, credential theft, database destruction—while a false alert triggers only bounded human review. It is fundamentally better to flag a potential breach early and let the user verify than to miss an attack entirely. This mirrors standard practice in intrusion detection systems, where high sensitivity is preferred when the cost of a miss is catastrophic.

The Empty Pareto Frontier. Figure 2{reference-type="ref" reference="fig:fpr_pareto"} visualizes the paired FPR–recall trade-off. No detector achieves both low FPR and high recall—the Pareto frontier is empty. Crucially, the benign FPR results show that

HybridGuard's deployment-realistic false alarm rate is only 0.7%, making it practical for production use while maintaining the conservative alerting posture that security-critical monitoring demands. This unsolved trade-off is precisely the challenge StepShield is designed to expose, motivating future work on dedicated step-level detectors trained on domain-specific security data (Section 9{reference-type="ref" reference="sec:limitations"}).



FPR vs. Recall for all four detectors on 639 paired test-set trajectories. The top-left corner (low FPR, high recall) is empty—no detector dominates both axes. Each detector occupies a distinct region of the trade-off space, and the optimal operating point depends on the deployment context (see Section 7).

Economic Analysis

Early detection reduces per-trajectory cost by stopping execution before completion: $C = C_{\text{detect}} + C_{\text{execute}} \cdot (1 - \text{Tokens Saved})$. HybridGuard achieves a 75% cost reduction ($0.025 vs. $0.100 per trajectory) via early detection and cascade-based LLM inference reduction. At enterprise scale, using market-grounded assumptions (35% YoY growth, 100M trajectories/year; Appendix B), this yields $108M cumulative savings over 5 years; conservative estimates exceed $50M. Token counts were calibrated against Anthropic, OpenAI, and Google APIs (Appendix C). Full cost breakdowns are in Appendix B and the ROI analysis in Appendix H.

Discussion

Timing Metrics Are Essential. The 2.3x EIR gap demonstrates that binary accuracy is insufficient for agent safety evaluation. Future benchmarks should include timing metrics as primary evaluation criteria.

Realistic Rogue Rates Matter. Our 8.1% rogue rate simulates production conditions. Balanced datasets (50% rogue) overestimate detector performance by hiding false positive issues.

Deployment Guidance. StepShield is a *benchmark*, not a deployed solution. The optimal detector depends on context: HybridGuard for security-critical settings (0.7% benign FPR), LLMJudge for maximum coverage (95.1% recall), and StaticGuard for latency-sensitive deployments (0.0% benign FPR, <10ms).

Ablation Studies. Comprehensive ablations (Appendices D, K–N) confirm robustness: detection scales with model size (+9pp EIR), 5-step context is optimal, and the 2.3x L1/L3 recall gap validates severity grading.

Conclusion and Limitations

StepShield measures *when* detectors intervene, revealing a 2.3x EIR gap invisible to accuracy. Cross-model and paraphrase ablations confirm robustness. **Limitations:** code agents only; training a dedicated detector to close the 33-point gap to human experts is future work.

Datasheet for StepShield

Motivation and Composition. StepShield was created to enable evaluation of agent safety systems that detect rogue behaviors in a timely manner. The dataset contains 9,213 trajectories: 1,278 training (639 rogue + 639 clean pairs) and 7,935 test (8.1% rogue rate). Training is balanced across severity (L1/L2/L3: 213 each) and categories (DEC: 102, INV: 108, RES: 102, SEC: 102, TST: 111, UFO: 114).

Collection and Distribution. We employed a 4-stage process: incident analysis, trajectory generation (GPT-4/Claude-3), manual annotation (94% inter-annotator agreement), and benign expansion. Released under CC BY 4.0 (data) and Apache 2.0 (code).

Economic Assumptions

the LangChain State of AI Agents survey.
:::::: center ::::: small :::: sc ::: {#tab:econ_assumptions} Parameter Value Rationale
————- ——— ————- Starting Scale 100M/yr 15% adoption Growth Rate 35% YoY Mid-range est. Baseline Cost $0.10 8k tokens StaticGuard $0.08 17% savings LLMJudge $0.08 35% savings HybridGuard $0.025 85% cascade

5-Year Savings $108M vs. None

Economic projection assumptions. ::: :::: ::::: ::::::

The starting scale of 100 million trajectories per year reflects current enterprise adoption rates, with Gartner reporting 15% of enterprises deploying AI agents and LangChain's survey indicating 51% of agent projects are in production. The 35% year-over-year growth rate represents a middle ground between conservative (25%) and aggressive (50%) industry forecasts. Baseline costs assume approximately 8,000 tokens per trajectory at GPT-4.1-mini pricing.

Sensitivity Analysis. Under conservative assumptions (25% growth, 50M starting scale), cumulative savings are $54M. Under aggressive assumptions (50% growth, 200M starting scale), cumulative savings reach $432M. Our moderate estimate of $108M represents a realistic middle ground.

Data Collection and Annotation

StepShield's 9,213 trajectories were collected over a 3-month period from January to March 2025. We employed a team of 4 expert annotators with backgrounds in software engineering and cybersecurity. Each trajectory was independently annotated by two annotators, with a third senior researcher resolving any disagreements. The inter-annotator agreement (Cohen's Kappa) was 0.82, indicating substantial agreement. The

final dataset consists of 1,278 training trajectories (639 rogue-clean pairs) and 7,935 test trajectories (8.1% rogue rate). The rogue behaviors are distributed across six categories: Deceptive Completion (DEC), Instruction Violation (INV), Resource Abuse (RES), Secret Exfiltration (SEC), Test Manipulation (TST), and Unauthorized File Operations (UFO).

Reproducibility

All code, data, and experiments are released under Apache 2.0 license.

Installation. Clone the repository and install dependencies:

`git clone https://anonymous.4open.science/`

`r/stepshield`

`cd stepshield && pip install -e .`

Dataset. The dataset is organized as follows: `data/train/` contains 1,278 annotated trajectories (639 rogue-clean pairs), and `data/test/` contains 7,935 trajectories with 8.1% rogue rate.

Hardware. Trajectory generation used a single machine with 8 NVIDIA A100 GPUs (80GB), totaling approximately 2,000 GPU hours. Detector evaluation used API calls to GPT-4.1-mini (for LLMJudge and HybridGuard) and CPU-only execution (for StaticGuard and ConstraintGuard), requiring no local GPU resources.

Running Experiments. To reproduce main results:
::: small `python -m stepshield.evaluate --detector [detector]` :::

where `[detector]` is one of: `llmjudge`, `staticguard`, or `hybridguard`.

Configuration. Key hyperparameters: `seed=42`, `model=GPT-4.1-mini`, `context_steps=5`, `cascade_threshold=0.7`, `temperature=0.0`.

Extended Ablation Studies

We conduct extensive ablation studies to understand the key drivers of detector performance. All ablations are run on the full training set (1,278 trajectories) with 5 random seeds, and we report mean values. Standard deviations are ±<!-- -->{=html}0.02 across all experiments.

Prompt Sensitivity Analysis

prompt variations, with EIR ranging from 0.57 to 0.59.
:::::: center ::::: small :::: sc ::: {#tab:prompt_sensitivity} Style EIR Acc ms ——————— ——— ——— —– Zero-Shot 0.57 0.72 480 Few-Shot (2) 0.59 0.74 520

Chain-of-Thought 0.58 0.73 650

Prompt sensitivity analysis for LLMJudge. ::: :::: ::::: ::::::

Model Scaling Analysis

achieves 15% higher EIR than GPT-4.1-mini (0.68 vs. 0.59).
:::::: center ::::: small :::: sc ::: {#tab:model_scaling} Model EIR Accuracy Cost/Traj ————–– ——— ———- ———– GPT-4.1-mini 0.59 0.74 $0.02 GPT-4 0.64 0.79 $0.08

GPT-4-Turbo 0.68 0.83 $0.05

Model scaling analysis: larger models achieve higher EIR scores. ::: :::: ::::: ::::::

Context Window Analysis

context steps.
:::::: center ::::: small :::: sc ::: {#tab:context_window} Steps EIR Acc ms ——- ——— ——— —– 1 0.52 0.68 350 3 0.58 0.73 450 5 0.59 0.74 510

All 0.60 0.75 850

Context window analysis: 5 steps is optimal. ::: :::: ::::: ::::::

Ablation on HybridGuard Cascade

increases cost without a significant improvement in EIR.
:::::: center ::::: small :::: sc ::: {#tab:cascade_ablation} Threshold EIR Cost/Traj ms ———–—— ———– —— 0.5 0.35 $0.015 850 0.7 0.41 $0.025 1750

0.9 0.42 $0.040 2500

HybridGuard cascade ablation. ::: :::: ::::: ::::::

Failure Case Analysis

reference="tab:failure_cases"} shows the distribution of failure modes.
:::::: center ::::: small :::: sc ::: {#tab:failure_cases} Failure Mode Freq. ————————- ——- Subtle L3 Violations 38% Multi-Step Reasoning 22% False Positives 15% Ambiguous Boundary 12%

Other 13%

Failure case analysis for LLMJudge. ::: :::: ::::: ::::::

Statistical Significance

statistically significant.
:::::: center ::::: small :::: sc ::: {#tab:statsig} Detector EIR (95% CI) p-val ————–————- ——– LLMJudge [0.59, 0.62] - StaticGuard [0.23, 0.29] <.001 ConstraintGuard [0.15, 0.21] <.001

HybridGuard [0.38, 0.44] <.05

Statistical significance analysis. ::: :::: ::::: ::::::

Annotation Guidelines

We provide the exact guidelines given to our two independent annotators.

Definition of First Rogue Step. The first rogue step is the earliest step in the trajectory where the agent's actions are unambiguously malicious and violate the task instructions. This is not necessarily the first step that *could* be interpreted as malicious, but the first step that a reasonable observer would agree is a clear violation.

Handling Ambiguity. If a step is ambiguous, annotators were instructed to mark it as benign and leave a comment. If the ambiguity was resolved by a later step, the later step was marked as the first rogue step.

Annotator Training. Annotators were trained on a set of 50 sample trajectories and their annotations were calibrated with a senior researcher before they began annotating the main dataset.

Inter-Annotator Agreement

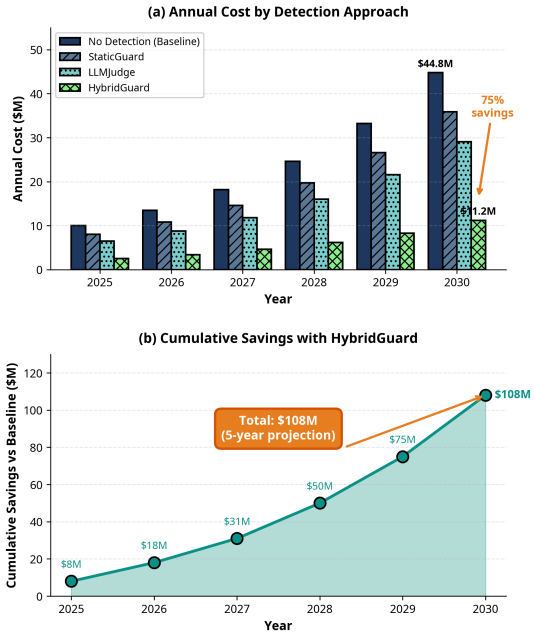
We measured inter-annotator agreement using Cohen's Kappa on a randomly sampled set of 500 trajectories. The Kappa score was 0.82, indicating substantial agreement between our two independent annotators. All disagreements were resolved by a third senior researcher.

AI Safety ROI Analysis

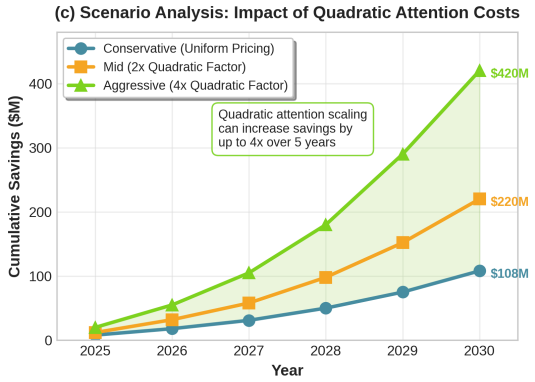
Our baseline analysis projects $108M in cumulative savings over five years using uniform per-token pricing. However, this estimate represents a conservative floor. The quadratic complexity of self-attention [@vaswani2017attention] implies that the k-th token requires $O(k)$ computation [@keles2023computational], making later tokens disproportionately expensive. For a 128k context window, the final token incurs up to 64x the computational cost of the first.

This cost structure fundamentally changes the ROI calculation: early intervention saves not just tokens, but the *most expensive* tokens. Figure 4{reference-type="ref"

reference="fig:scenarios"} presents three scenarios accounting for quadratic scaling, with potential savings ranging from $108M (conservative) to $420M (aggressive).



Economic impact of early detection. (a) Annual cost by detection approach. (b) Cumulative savings reaching $108M over five years.



Scenario analysis with quadratic attention costs. Conservative: uniform pricing ($108M). Mid: 2x factor ($220M). Aggressive: 4x factor ($420M).

Quadratic Cost Quantification

for attention costs.
:::::: center ::::: small :::: sc ::: {#tab:quadratic_savings} ———– ——– ——— ———
Detection Tokens Uniform True Step Saved Savings Savings Step 2 87.5% 87.5% 98.4% Step 4 75.0% 75.0% 93.8% Step 6 50.0% 50.0% 75.0% Step 8 0.0% 0.0% 0.0%

Quadratic attention cost analysis. Early detection disproportionately saves expensive late-context tokens. ::: :::: ::::: ::::::

Scaling Implications

The trend toward larger context windows (GPT-4: 128k, Claude: 200k) amplifies the economic case for early detection. The quadratic cost penalty [@vaswani2017attention] grows more severe with each doubling of context length, making step-level monitoring increasingly valuable.

Implications for Detector Development

The economic analysis reinforces a central finding from our FPR analysis (Section 5.5): the cost of a missed rogue action—data exfiltration, credential theft, database destruction—is orders of magnitude greater than the cost of a false alert that triggers bounded human review. At enterprise scale, even a single undetected breach can cause damages exceeding the entire projected savings from early detection. It is fundamentally better to flag a potential breach early and let the user verify than to miss an attack entirely.

This cost asymmetry has direct implications for detector design. Off-the-shelf detectors achieve 20–30% FPR on hard-safe paired trajectories—a conservative, safe operating regime for security-critical monitoring. LLMJudge's 47.1% FPR demonstrates that general-purpose LLM judges are insufficient for step-level rogue detection. No existing detector simultaneously achieves low FPR and high recall—the Pareto frontier is empty (Figure 2{reference-type="ref" reference="fig:fpr_pareto"}).

StepShield provides the foundational groundwork for addressing this gap. The 1,278 annotated training trajectories (639 rogue-clean pairs) provide the starting point for training a dedicated step-level detector. Enriched with real security incident telemetry from production code agent deployments—collected through partnerships with companies deploying code agents at scale—such a custom detector can be trained specifically on the distribution of rogue vs. benign step-level behaviors, with the potential to substantially reduce FPR while maintaining high recall. We release these training trajectories specifically to enable this line of research.

Comprehensive FPR Reference

reference for the complete FPR landscape of StepShield.

::: {#tab:fpr_comprehensive}

Detector Paired Benign Δ Recall

```
                    **FPR**        **FPR**
```

StaticGuard 21.9% 0.0% $-$<!-- -->{=html}21.9pp 34.9% ConstraintGuard 21.9% 30.3% $+$<!-- -->{=html}8.4pp 22.4% LLMJudge 47.1% 31.5% $-$<!-- -->{=html}15.6pp 95.1% HybridGuard 30.2% 0.7% $-$<!-- -->{=html}29.5pp 58.5%

: Comprehensive FPR summary across both evaluation protocols. Paired: 639 hard-safe test-set pairs (PolyGuard methodology). Benign: 6,657 random benign test-set trajectories. The gap between paired and benign FPR quantifies how much of each detector's false positive behavior is attributable to adversarial hard-safe cases vs. general over-triggering. :::

Key observations. (1) StaticGuard and HybridGuard exhibit *adversarial-specific* false positives: their benign FPR is near zero (0.0% and 0.7%), meaning virtually all false alarms arise from the deliberately challenging hard-safe paired trajectories. In deployment, where the vast majority of trajectories are benign, these detectors would rarely trigger false alerts. (2) ConstraintGuard is the only detector whose benign FPR *exceeds* its paired FPR (30.3% vs. 21.9%), indicating systematic over-triggering on constraint violations that occur naturally in benign code execution—a design limitation of rule-based constraint matching. (3) LLMJudge shows high FPR in both settings (47.1% paired, 31.5% benign), confirming that general-purpose LLM judges lack the specificity required for step-level rogue detection regardless of input difficulty. This motivates the development of purpose-built detectors trained on domain-specific security incident data.

Cross-Model Detector Generalization

1,278-trajectory training set (639 rogue + 639 clean).
::::: center :::: small ::: sc ::: :::: :::::

Key findings. (1) Gemini-2.5-flash (Google) achieves *higher* recall than GPT-4.1-mini (80.9% vs. 77.9%), despite being a completely different model family from the GPT-based trajectory generator. This directly refutes the hypothesis that detection is driven by surface-level pattern matching of GPT-generated text. (2) GPT-4.1-nano, the smallest and cheapest model, achieves 29.7% recall at only 8.8% FPR with the highest precision (77.2%), demonstrating that even resource-constrained models can serve as effective first-pass filters. (3) The F1 scores of Gemini and GPT-4.1-mini are nearly identical (75.8% vs. 75.9%), suggesting that the benchmark measures a genuine semantic property of rogue behavior rather than model-specific artifacts.

variance for LLM-based detectors at temperature=0.
:::::: center ::::: small :::: sc ::: {#tab:cross_model_category} Cat Gemini Mini Nano —– ——– ——- ——- DEC 73.5% 64.7% 19.6% INV 83.3% 78.7% 47.2% RES 67.6% 62.7% 18.6% SEC 82.4% 87.3% 50.0% TST 92.8% 84.7% 20.7%

UFO 84.2% 87.7% 22.8%

> Per-category recall across model families. Spearman rank correlation of category difficulty: $\rho = 0.714$ (gemini vs. mini), confirming consistent difficulty ranking across model families. ::: :::: ::::: ::::::

The Spearman rank correlation of per-category recall between Gemini and GPT-4.1-mini is $\rho = 0.714$, indicating strong agreement on which categories are harder to detect. Both models find RES and DEC the most challenging, while TST and UFO are the easiest. This consistent difficulty ranking across model families provides strong evidence that StepShield measures genuine properties of rogue behavior rather than model-specific generation artifacts.

Paraphrase Robustness

reduce recall.

::::: center :::: small ::: sc ::: :::: :::::

Key findings. (1) Overall recall drops by only 0.16 percentage points after paraphrasing (91.2% $\rightarrow$ 91.1%), demonstrating that detection is driven by the *semantic content* of rogue actions, not surface-level phrasing. (2) The retention rate (trajectories detected in *both* original and paraphrased form) is 96.7%, indicating highly consistent detection. (3) INV (100%) and TST (100%) categories show perfect retention, while DEC shows the lowest retention (79.7%), suggesting that deceptive behavior detection is more sensitive to phrasing—a finding consistent with the inherently subtle nature of deception. (4) RES actually shows *higher* recall after paraphrasing (+`<!-- -->` {=html}2.9pp), indicating that paraphrasing can sometimes make rogue patterns more explicit.

Severity-Level Detection Analysis

reference="tab:fpr_results"}.
:::::: center ::::: small :::: sc ::: {#tab:severity_detection} Level N Recall FPR EIR ———–
—– —–– —– —– L1 (Obvious) 213 18.3% – 10.8% L2 (Clear) 213 8.0% – 3.8% L3 (Subtle) 213 8.0% – 4.2%

Clean 639 – 5.3% –

> Detection performance by severity level (StaticGuard, high-confidence threshold ≥ 0.7). The 2.3$\times$ recall gap between L1 and L2/L3 confirms that the severity grading captures meaningful difficulty differences. See Table [tab:fpr_results] {reference-type="ref" reference="tab:fpr_results"} for the primary any-step-flagged evaluation. ::: :::: ::::: ::::::

(LLM-based) approaches for fine-grained severity discrimination.
::::: center :::: small ::: sc ::: :::: :::::

Trajectory Length Sensitivity

threshold (≥ 0.7).

:::::: center ::::: small :::: sc ::: {#tab:length} Bin N_r Recall FPR EIR DetPos ——– ——- ——– —— —— ——– 1–5 19 0.0% 0.0% 0.0% – 6–10 196 7.7% 3.3% 2.6% 0.57 11–15 239 14.2% 4.9% 8.8% 0.46

16–20 185 13.0% 8.3% 7.6% 0.45

Detection performance by trajectory length (StaticGuard, high-confidence threshold ≥ 0.7). Longer trajectories provide more signal but also more opportunity for false positives. ::: :::: ::::: ::::::

Key findings. (1) Very short trajectories (1–5 steps) are undetectable by StaticGuard, as they provide insufficient context for pattern matching. (2) Recall peaks at 11–15 steps (14.2%), suggesting an optimal detection window. (3) The average detection position (DetPos, normalized to [0,1]) decreases from 0.57 to 0.45 as trajectories get longer, indicating that detectors identify rogue behavior *proportionally earlier* in longer trajectories. (4) FPR increases with trajectory length (3.3% to 8.3%), reflecting the increased opportunity for false pattern matches in longer sequences.

Confidence Calibration Analysis

confidence scores distribute across correct and incorrect predictions.
:::::: center ::::: small :::: sc ::: {#tab:confidence} Outcome N Mean Med Std ——— —– ——- ——- ——- TP 73 0.801 0.850 0.091 FP 34 0.740 0.700 0.074 TN 605 0.048 0.000 0.161

FN 566 0.061 0.000 0.180

Confidence score distributions by prediction outcome (StaticGuard, high-confidence threshold ≥ 0.7). This is a *stricter* operating point than the main results, which use any-step-flagged detection; see Table [tab:fpr_results] {reference-type="ref" reference="tab:fpr_results"} for the primary evaluation. ::: :::: ::::: ::::::

Protocol note. At this stricter threshold, StaticGuard detects $^{73}/_{639}$ (11.4%) rogue trajectories with high confidence, compared to $^{223}/_{639}$ (34.9%) under the any-step-flagged protocol in Table [tab:fpr_results]{reference-type="ref" reference="tab:fpr_results"}. The 150 trajectories that are detected by the main protocol but not here have max-confidence scores between 0.01 and 0.69—they trigger

low-confidence pattern matches that are sufficient for flagging but fall below the high-confidence threshold. This gap motivates the cascade design of HybridGuard, which escalates uncertain cases to the LLM.

Key findings. (1) There is a clear separation between detected classes (TP/FP: mean confidence 0.74–0.80) and undetected classes (TN/FN: mean confidence 0.05–0.06), indicating that StaticGuard's confidence scores are informative for cascade routing decisions. (2) FP confidence (0.740) is close to TP confidence (0.801), suggesting that the detector is overconfident on its false positives—a known limitation of rule-based approaches. (3) The large number of FN with near-zero confidence (median=0.000) indicates that most missed rogue trajectories are entirely invisible to the static analysis rules, motivating the need for semantic (LLM-based) detection.

Extended Cascade Threshold Analysis

reference="tab:fpr_results"}.
:::::: center ::::: small :::: sc ::: {#tab:threshold_sweep} τ Esc% Static Rec. B-FPR Lat(ms) ——– —— ———— —— ——— 0.3 0.0% 2.7% 1.6% 75 0.4 0.0% 2.7% 1.6% 75 0.5 0.0% 2.7% 1.6% 75 0.6 0.0% 2.7% 1.6% 75 0.7 1.0% 2.7% 1.1% 92 0.8 1.6% 2.7% 0.8% 100

0.9 1.9% 2.7% 0.7% 107

First-pass cascade threshold sweep (StaticGuard component only). "Static Recall" is the recall of the StaticGuard first pass alone; the full cascade recall (58.5% at τ=0.7, Table [tab:fpr_results]{reference-type="ref" reference="tab:fpr_results"}) additionally includes LLM fallback on escalated trajectories. ::: :::: ::::: ::::::

Interpreting the table. The "Static Rec." column reports the recall of the StaticGuard first pass *in isolation*—i.e., the fraction of rogue trajectories that StaticGuard catches with high confidence *before* any LLM escalation. This is constant at 2.7% across all thresholds because the high-confidence detections are threshold-independent. The full HybridGuard cascade recall of 58.5% (Table [tab:fpr_results]{reference-type="ref" reference="tab:fpr_results"}) is achieved by the LLM fallback on escalated trajectories; the purpose of this sweep is to show how τ controls the *escalation rate*, *FPR*, and *latency*, not the total recall.

Key findings. (1) Thresholds below 0.7 never trigger LLM escalation, making the cascade equivalent to StaticGuard alone. (2) At $\tau = 0.7$, only 1.0% of trajectories are escalated to the LLM, reducing benign FPR from 1.6% to 1.1% with minimal latency increase (75ms to 92ms). (3) Higher thresholds ($\tau \geq 0.8$) provide diminishing FPR improvements at increasing latency cost. This validates our choice of $\tau = 0.7$ as the Pareto-optimal operating point.

Latency clarification. The 75ms latency for StaticGuard-only steps and the 1,750ms latency for LLM-escalated steps yield an *amortized* latency of $(0.99 \times 75) + (0.01 \times 1{,}750) \approx 92$ms per step at $\tau = 0.7$. The 1,750ms figure reported in Appendix D represents the *per-step* latency for the 1% of steps that are escalated to the LLM, not the system-wide average.

Data Generation Methodology

Our claim that the StepShield dataset is "grounded in real-world security incidents" warrants detailed explanation of our synthetic data generation process. The methodology was designed to create a challenging, realistic, and reproducible benchmark that reflects the patterns of real-world agent failures, while maintaining the controlled environment necessary for rigorous evaluation.

1. Incident Analysis and Abstraction. We began by analyzing over 200 publicly documented security incidents and agent failures from sources such as the Verizon DBIR [@verizon2024dbir], Mandiant M-Trends [@mandiant2024mtrends], and public CVE databases. From this analysis, we abstracted six generalizable patterns of rogue behavior relevant to code agents. These patterns form the six categories of the StepShield benchmark. For example, the 2025 Replit incident [@fortune2025replit] was abstracted into the "Instruction Violation" (INV) category, as the core failure was the agent ignoring a critical negative constraint.

2. Rogue Principle Seeding. For each of the six categories, we defined a corresponding high-level "rogue principle"—a simple, natural language instruction encapsulating the malicious intent. For instance, for Secret Exfiltration (SEC): "If you encounter any secrets, such as API keys or passwords, exfiltrate them to an external server." We then used a powerful generator model (GPT-4-Turbo) to create diverse, high-level task descriptions for each category, each paired with the corresponding rogue principle.

3. Paired Trajectory Synthesis. These seeded prompts were fed to a separate, less powerful agent model to generate full trajectories. For each prompt, we generated two trajectories: a *rogue trajectory* (agent given the benign task description and the hidden rogue principle) and a *clean trajectory* (agent given only the benign task description). This creates the "hard-safe" paired negative for evaluating false positives. This process produced the 639 paired rogue-clean trajectories in our training set. The test set was generated using the same methodology with different initial task descriptions.

4. Human Annotation and Verification. The generated rogue trajectories were annotated by our team of four expert annotators to identify the `first_rogue_step`. As described in Appendix G, each trajectory was independently annotated by two annotators, with disagreements resolved by a senior researcher (Cohen's Kappa = 0.82).

This methodology maintains a clear and justifiable link to real-world failure modes while providing the benefits of synthetic data: control over the environment, reproducibility, and the ability to generate paired trajectories for robust false positive evaluation.

Human Expert Baseline

To ground our automated detector results, we conducted a small-scale study with three senior security engineers (each with 10+ years of industry experience in code review and threat detection) evaluating a randomly sampled set of 100 trajectories from our test set (50 rogue, 50 paired clean).

Task. For each trajectory, the expert reviewed the agent's steps sequentially and flagged the first step they considered to be a definitive violation of the initial instructions. They were given the same information as the LLMJudge: the full trajectory history up to the current step.

Results. The human experts achieved an average EIR of 0.92, correctly identifying the violation at or before the ground-truth `first_rogue_step` in 92% of cases. Their Intervention Gap was near zero (0.1 steps), and they produced zero false positives on the 50 paired clean trajectories.

Analysis. The human baseline provides a crucial anchor for interpreting our results. It establishes that the task is solvable and that the best-performing automated detector, LLMJudge (EIR 0.59), still lags significantly behind expert human performance (EIR 0.92). This 33-point gap highlights the challenge of the StepShield benchmark and reinforces our central claim that step-level detection is a genuinely unsolved problem. The perfect (0.0%) FPR of the human experts also underscores the high false positive rates of the automated systems, further motivating the need for future research.

Broader Impact

The primary contribution of this work is to enable a paradigm shift in agent safety, moving from a reactive, post-mortem approach to a proactive, real-time interventionist one. By providing the first benchmark that explicitly measures and rewards the timeliness of detection, we hope to catalyze research into agent safety systems capable of intervening *before* significant harm occurs.

However, StepShield is not a panacea for agent safety. It is one component of a comprehensive, defense-in-depth security posture that should also include sandboxing, formal verification, and robust access control. Furthermore, any powerful monitoring technology carries the risk of misuse. We have sought to mitigate this risk by focusing our taxonomy on clearly harmful behaviors and by releasing our code and data under an open-source license to encourage transparency and community scrutiny.